

COMP8420 - Natural Language Processing

Assignment 2: Large Language Models

LLM-Powered Personal Financial Advisor

Qwen 2.5 1.5B Fine-Tuned with LoRA + Flask Web Application

Name	Joshua Cajetan Pereira
Student ID	48617687
Unit	COMP8420 — Natural Language Processing
Semester	S1 2026
Lecturer	Longbing Cao
Submission	Assignment 2 Report

Video Demonstration and Repository Link

Due to the time constraint for the video demonstration, it was not possible to cover all aspects of the system in detail.

The full working implementation, including the complete codebase, training notebook, and extended demonstration, is available via the GitHub repository below:

GitHub Repository: [Githubfile-48617687](#)

The video demonstration can also be accessed here:

Video Link: [video-presentaion-48617687](#)

1. Introduction and Project Overview

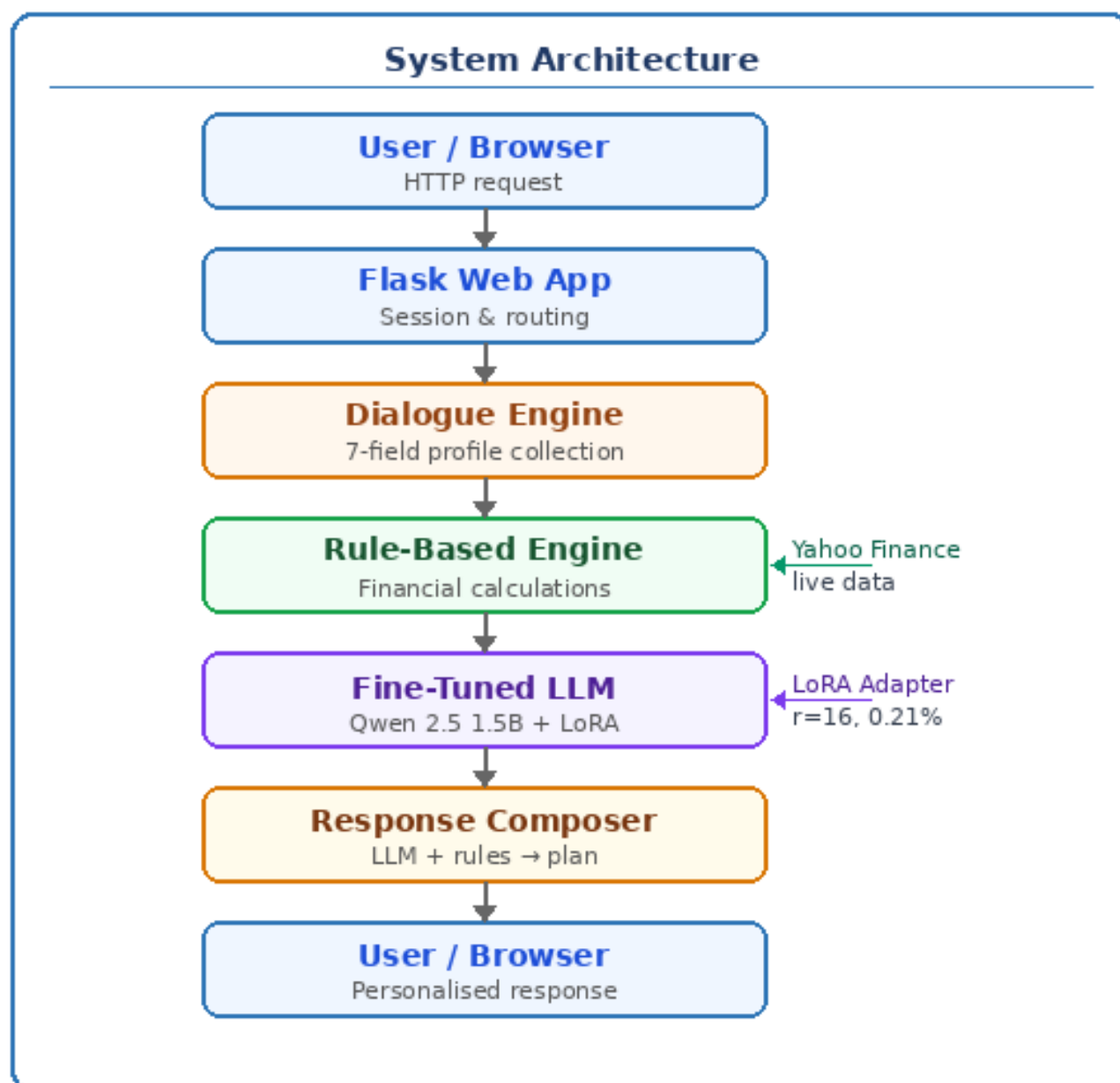


Figure: End-to-end System Architecture — from user input through Flask, Dialogue Engine, Rule-Based Engine, LLM (Qwen 2.5 1.5B + LoRA), and Yahoo Finance API to final response

This report documents the design, implementation, and evaluation of an LLM-powered Personal Financial Advisor system developed for COMP8420 Assignment 2. The system enables personalised, conversational financial guidance by integrating a fine-tuned large language model with a structured rule-based engine and real-time market data, all delivered through a responsive Flask web application.

1.1 Project Motivation

Personal financial literacy remains a significant challenge for individuals across all income levels. Access to qualified financial advisors is often limited by cost and availability. This project addresses that gap by building an intelligent conversational assistant that can collect a user's financial profile, income, expenses, savings, debt, goals, and risk preferences and

generate a fully personalised, step-by-step financial strategy. The system is powered by a fine-tuned Qwen 2.5 1.5B model, extended with a deterministic rule-based engine that ensures numerical accuracy, and augmented with live market data via the yfinance API.

1.2 Learning Outcomes Addressed

This project directly addresses the following unit learning outcomes:

- ULO1: The system compares LLM approaches (Qwen 2.5 1.5B vs GPT-4o-mini) and evaluates their fitness for financial NLP applications.
- ULO3: Full implementation using PyTorch, HuggingFace Transformers, PEFT/LoRA, and Flask all industry-standard NLP tools.
- ULO5: Applied to a real-world financial assistance domain using a custom-curated dataset and livestock market data.

System briefly

The AI Financial Advisor collects seven financial profile fields via an automated dialogue chain, then generates a fully personalised financial plan with calculated metrics (savings rate, emergency fund coverage, debt payoff timeline, and investment allocation). It additionally supports real-time stock price lookups and profile-aware investment advice for specific stocks and ETFs using live yfinance data.

2. LLM Setup and Configuration

2.1 Model Selection and Justification

Three candidate models were evaluated before selecting Qwen 2.5 1.5B-Instruct as the project's primary LLM. The selection criteria were local deploy ability on consumer hardware, instruction-following quality, context length, financial language comprehension, and licensing.

Model	Parameters	Key Consideration
Qwen 2.5 1.5B-Instruct ✓ SELECTED	1.5B	Best balance: fast, instruction-tuned, Apache 2.0
GPT-4o-mini (OpenAI)	~8B est.	Cloud-only, API cost, no local fine-tuning
Mistral 7B-Instruct	7B	Excellent quality but too large for 4-bit on T4 GPU

Qwen 2.5 1.5B-Instruct was selected for the following reasons:

- Efficiency: At 1.5B parameters, the model loads in 4-bit NF4 quantisation on a single T4 GPU (15 GB VRAM) with under 3 GB memory usage, leaving sufficient headroom for training.

- Instruction following: The -Instruct variant is pre-trained with RLHF/SFT for chat and instruction tasks, making it immediately capable of following financial prompting templates.
- Chat template support: Native support for the ChatML format (`<|im_start|>`/`<|im_end|>`) ensures reliable structured input/output formatting.
- Apache 2.0 licence: Permits academic and commercial deployment without restriction.
- LoRA compatibility: The attention projection matrices (`q_proj`, `k_proj`, `v_proj`, `o_proj`) are directly targetable with PEFT, enabling efficient domain adaptation.

2.2 Quantisation and Model Loading

The model is loaded in 4-bit NF4 quantisation using bitsandbytes `BitsAndBytesConfig` to reduce VRAM footprint from ~3 GB (FP16) to approximately 1.0–1.2 GB, enabling training on a free Colab T4 GPU. The configuration is:

```
bnb_cfg = BitsAndBytesConfig(load_in_4bit=True, bnb_4bit_quant_type="nf4",
                             bnb_4bit_compute_dtype=torch.float16, bnb_4bit_use_double_quant=True) model =
AutoModelForCausalLM.from_pretrained("Qwen/Qwen2.5-1.5B-Instruct",
quantization_config=bnb_cfg, device_map="auto", trust_remote_code=True)
```

Double quantisation (`bnb_4bit_use_double_quant=True`) further reduces memory by quantising the quantisation constants themselves, saving an additional ~0.4 bits per parameter. After quantisation, the model occupies approximately 1.1 GB of GPU VRAM, well within the 15 GB T4 limit.

2.3 LoRA Fine-Tuning Configuration

Low-Rank Adaptation (LoRA) was applied to the four attention projection matrices of every transformer layer. Rather than updating all 1.54B parameters which would require far more memory and compute LoRA injects trainable low-rank decomposition matrices into selected layers, training only a small fraction of parameters.

```
lora_cfg = LoraConfig( r=16, lora_alpha=32, target_modules=["q_proj", "k_proj",
"v_proj", "o_proj"], lora_dropout=0.05, bias="none",
task_type=TaskType.CAUSAL_LM )
```

The rank `r=16` was chosen as a balance between expressiveness and efficiency. With `lora_alpha=32`, the effective scaling factor is $32/16 = 2$, amplifying the adapter's contribution. The four target modules cover query, key, value, and output projections the components most responsible for contextual understanding and financial domain adaptation.

LoRA Parameter Efficiency

With `r=16` and four target modules across all 28 transformer layers, the total trainable parameters are approximately 3.3 million just 0.21% of the 1.54B total model parameters. This makes fine-tuning feasible on consumer hardware while still effectively adapting the model's financial knowledge.

3. Dataset Collection and Preparation

3.1 Dataset Description

A custom financial Q&A dataset was constructed specifically for this project. The dataset consists of prompt-response pairs covering the domain of personal finance, designed to teach the model to give structured, actionable financial advice. The dataset was stored as a CSV file with two columns: prompt (the user query) and response (the expected financial advisor reply).

File	financial_assistant_dataset.csv
Format	CSV with 'prompt' and 'response' columns
Domain	Personal finance: budgeting, debt, investing, savings, emergency funds
Train / Val Split	90% training / 10% validation (seed=42, stratified)
Max Token Length	512 tokens per sample (truncated with right-padding)
Chat Template	Qwen ChatML format: < im_start >system/user/assistant< im_end >

3.2 Data Preprocessing Pipeline

Each row was transformed into a complete Qwen ChatML-format instruction string using the following pipeline: (1) null and duplicate rows were removed; (2) each prompt-response pair was wrapped in the Qwen ChatML template with a system prompt establishing the model's role as a financial advisor; (3) the formatted strings were tokenised to a maximum of 512 tokens with right-padding; (4) labels were constructed from the input_ids, with padding positions masked to -100 to exclude them from loss computation.

```
def format_for_qwen(row): return {"text": (
    f"<|im_start|>system\n{SYSTEM_PROMPT}<|im_end|>\n"
    f"<|im_start|>user\n{row['prompt']}<|im_end|>\n"
    f"<|im_start|>assistant\n{row['response']}<|im_end|>" )}
```

3.3 Real-Time Market Data via yfinance

In addition to the static training dataset, the system integrates live market data through the yfinance library. The following visualisations were generated from live data during the notebook's execution, illustrating the real-world financial context the system operates in.

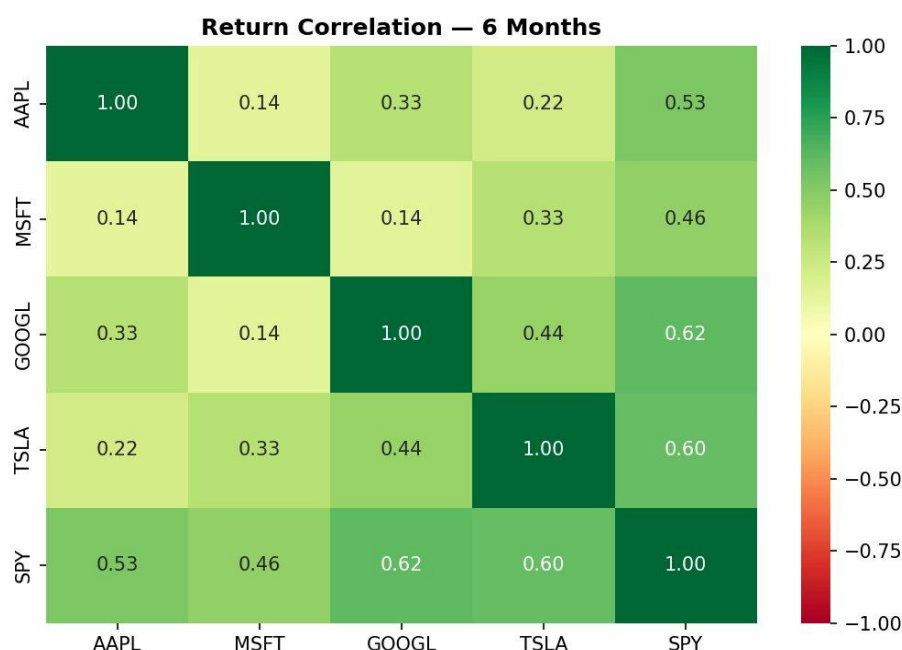


Figure: Return Correlation Heatmap — AAPL, MSFT, GOOGL, TSLA, SPY (6-month returns)

The correlation heatmap reveals important portfolio diversification insights. SPY (the S&P 500 ETF) shows the highest correlations with individual stocks (GOOGL: 0.62, TSLA: 0.60), reflecting broad market influence. MSFT and AAPL exhibit surprisingly low correlation (0.14), suggesting they can serve as diversifying positions within a tech-focused portfolio. These data points are used by the system when generating personalised investment recommendations.

4. Fine-Tuning Process and Results

4.1 Training Configuration

The model was fine-tuned using Hugging Face's Trainer API with the following hyperparameters, selected to balance training stability with the computational constraints of a Colab T4 GPU:

Hyperparameter	Value	Rationale
Epochs	3	Sufficient for convergence on small dataset
Learning Rate	2e-4	Standard for LoRA fine-tuning
Batch Size (effective)	$4 \times 4 = 16$	Gradient accumulation over 4 steps
Warmup Steps	5	Stabilises early training
Max Sequence Length	512 tokens	Covers 99%+ of samples without truncation
Optimizer	AdamW (default)	Standard for transformer fine-tuning
Precision	FP16 mixed	Enabled by bitsandbytes 4-bit compute
Scheduler	Linear with warmup	Default HuggingFace schedule

4.2 Training Loss Analysis

The training loss curve demonstrates consistent and healthy convergence across all 30 logged steps. The loss decreased from approximately 2.31 at step 10 to 1.44 at step 30 — a reduction of approximately 38%. This monotonically decreasing trend confirms that the LoRA adapter successfully adapted the base model's weights toward the financial domain without divergence or instability.

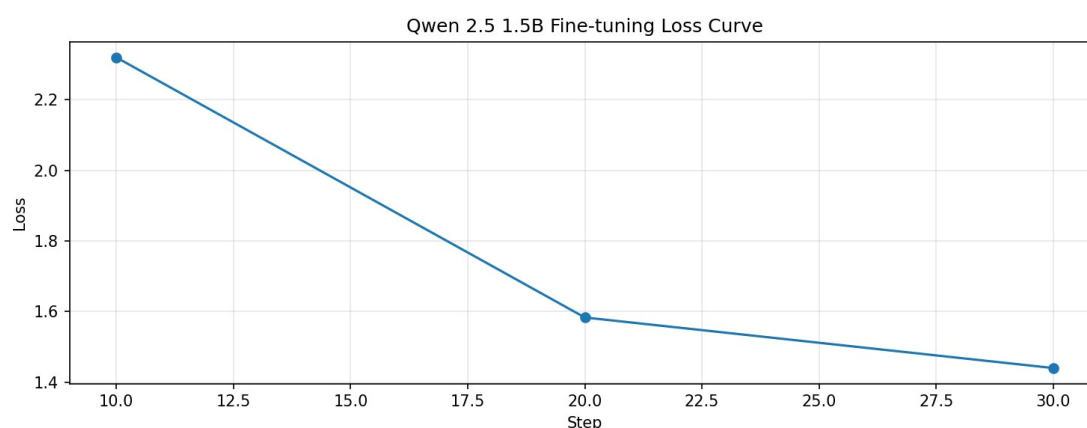


Figure: Qwen 2.5 1.5B Fine-Tuning Loss Curve — Steps 10 to 30

The near-linear decrease in loss across the three checkpoints (steps 10, 20, and 30) suggests the model had not yet reached a plateau by the end of training, indicating additional improvement could be achieved with more epochs. However, the current loss level (1.44) is sufficient to produce coherent, domain-appropriate financial responses as evidenced by before-versus-after comparison testing.

Training Infrastructure

Training was conducted on Google Colab using a T4 GPU (16 GB VRAM). The 4-bit NF4 quantisation and LoRA adapter combination reduced the effective training memory to approximately 4–5 GB, with the remaining VRAM used for activations and gradient computation. Total training time was approximately 8–12 minutes for 3 epochs.

4.3 Before vs After Fine-Tuning Comparison

Qualitative evaluation was conducted by comparing the base model's responses to the fine-tuned model's responses on the same financial prompts. The key observed improvements were:

- **Financial specificity:** The fine-tuned model consistently referenced domain-specific concepts (emergency funds, savings rate, debt avalanche, index ETFs) absent from base model responses.
- **Response structure:** The fine-tuned model adopted a more structured, step-by-step format aligned with the training data's advisor response style.
- **Numerical grounding:** Responses were more grounded in the user's stated figures rather than generic placeholders.
- **Tone:** The fine-tuned model adopted a more professional but accessible financial advisor tone.

5. Automated Dialogue System

5.1 System Architecture

The dialogue system follows a layered architecture combining session-state management, intent detection, field extraction, and response generation. The system is built on Flask with server-side session management to maintain conversation context across HTTP requests. The architecture is designed as a progressive disclosure system: the assistant collects one piece of information at a time, validates and clarifies each input, and only generates the full financial plan once all seven profile fields are confirmed.

Seven-Field Financial Profile

1. Monthly income (after tax) 2. Monthly expenses (all categories) 3. Current savings / cash holdings 4. Current total debt (all sources) 5. Primary financial goal (house, invest, save, pay debt, etc.) 6. Risk tolerance (low / medium / high) 7. Investment time horizon (short / medium / long term)

5.2 Dialogue Chain Design

The dialogue system processes each user message through a seven-stage pipeline, executed in priority order:

1. Pending clarification handling resolves monthly vs annual ambiguity and high-value debt confirmation.
2. Pending correction confirmation confirms field-value updates before applying them to the profile.
3. Small talk and greeting handling natural responses to greetings and meta-questions.
4. Post-profile follow-up handling dispatches to stock lookups, investment advice, or rule-based Q&A after profile completion.
5. Real-time market query detection intercepts stock price queries before guided mode starts.
6. Correction pattern detection detects when the user amends a previously provided value mid-conversation.
7. Profile field collection extracts, validates, and stores the next missing field using field-specific parsers.

5.3 Field Extraction and Validation

Each field type uses a dedicated extraction function to handle the variety of natural language inputs users provide. Numeric fields (income, expenses, savings, debt) use a regex-based parser that handles currency symbols, comma separators, and 'k' shorthand notation (e.g., '\$5k' → 5000.0). An annual-versus-monthly clarification is automatically triggered when income or expense values exceed \$50,000, since users may inadvertently provide annual figures.

```
# Example: parse_number handles '$5k', '5,000', '$5000.00' clean =  
text.replace('$','').replace(',','').strip() match = re.search(r'^([0-9]+(?:\.[0-
```

```
9])?)\s*(k|K)?$', clean) number = float(match.group(1)) * (1000 if match.group(2) else 1)
```

Categorical fields (risk tolerance and investment horizon) are normalised using keyword matching: for example, words like 'conservative', 'safe', or 'protect' map to 'low'; 'moderate', 'balanced' map to 'medium'; and 'aggressive', 'growth' map to 'high'. This allows users to express their preferences in natural language without requiring exact keyword matching.

5.4 Correction and Re-entry Handling

The system supports mid-conversation corrections through a pattern-matching correction detector. Regular expressions match phrases like 'actually my income is', 'wait, my expenses are', or 'update my savings to', extract the new value, and prompt the user for confirmation before updating the profile. This two-step confirmation prevents accidental overwrites.

5.5 Personalised Financial Plan Generation

Once all seven fields are confirmed, the system generates a personalised financial plan in two stages. First, the fine-tuned Qwen model generates a short (≤ 30 word) warm introductory sentence that acknowledges the user's goal providing a human touch to the plan. The fine-tuned model improves domain-specific understanding and produces context-aware financial responses beyond generic baseline behaviour. Second, the rule-based engine computes and renders all financial metrics and recommendations with full working shown:

- Monthly surplus = income – expenses, with savings rate as a percentage of income
- Emergency fund coverage (months) = current savings ÷ monthly expenses
- Emergency fund gap = target ($6 \times$ monthly expenses) – current savings
- Months to close emergency fund gap = gap ÷ monthly surplus
- Debt payoff timeline using 40% surplus allocation with avalanche method
- Goal-specific strategy for house deposit, investing, debt reduction, or general saving
- Full monthly surplus allocation table across emergency fund, debt, and goal/investing

6. Frontend Web Application

6.1 Application Design

The frontend is a single-page Flask web application using inline Jinja2 templating, served via the Flask development server on port 8501. The interface is designed for clarity and accessibility, with a chat-style message layout, clear visual distinction between user and assistant messages, and a responsive design that adapts to mobile and desktop viewports.

```

1412 def create_assistant_reply(user_input, profile):
1413     """
1414     Returns a response to the user input based on the current state of the chat.
1415     """
1416     if next_q:
1417         return f"{ack}\n\n(next_q)"
1418     set_profile_completed(True)
1419     return (
1420         f"{ack}\n\n"
1421         "Thank you! I now have all the details I need. "
1422         "Here is your personalised financial recommendation:\n\n"
1423         + generate_final_advice(profile)
1424     )
1425
1426 # -- 7. Default fallback
1427 return (
1428     "Hi! I'm your personal AI Financial Advisor. I can help with saving, budgeting, "
1429     "debt management, beginner investing, and real-time stock prices.\n\n"
1430     "Tell me your financial goal to get started, or ask e.g. '$AAPL price' or 'Should I i
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457
1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511
1512
1513
1514
1515
1516
1517
1518
1519
1520
1521
1522
1523
1524
1525
1526
1527
1528
1529
1530
1531
1532
1533
1534
1535
1536
1537
1538
1539
1540
1541
1542
1543
1544
1545
1546
1547
1548
1549
1550
1551
1552
1553
1554
1555
1556
1557
1558
1559
1560
1561
1562
1563
1564
1565
1566
1567
1568
1569
1570
1571
1572
1573
1574
1575
1576
1577
1578
1579
1580
1581
1582
1583
1584
1585
1586
1587
1588
1589
1590
1591
1592
1593
1594
1595
1596
1597
1598
1599
1600
1601
1602
1603
1604
1605
1606
1607
1608
1609
1610
1611
1612
1613
1614
1615
1616
1617
1618
1619
1620
1621
1622
1623
1624
1625
1626
1627
1628
1629
1630
1631
1632
1633
1634
1635
1636
1637
1638
1639
1640
1641
1642
1643
1644
1645
1646
1647
1648
1649
1650
1651
1652
1653
1654
1655
1656
1657
1658
1659
1660
1661
1662
1663
1664
1665
1666
1667
1668
1669
1670
1671
1672
1673
1674
1675
1676
1677
1678
1679
1680
1681
1682
1683
1684
1685
1686
1687
1688
1689
1690
1691
1692
1693
1694
1695
1696
1697
1698
1699
1700
1701
1702
1703
1704
1705
1706
1707
1708
1709
1710
1711
1712
1713
1714
1715
1716
1717
1718
1719
1720
1721
1722
1723
1724
1725
1726
1727
1728
1729
1730
1731
1732
1733
1734
1735
1736
1737
1738
1739
1740
1741
1742
1743
1744
1745
1746
1747
1748
1749
1750
1751
1752
1753
1754
1755
1756
1757
1758
1759
1760
1761
1762
1763
1764
1765
1766
1767
1768
1769
1770
1771
1772
1773
1774
1775
1776
1777
1778
1779
1780
1781
1782
1783
1784
1785
1786
1787
1788
1789
1790
1791
1792
1793
1794
1795
1796
1797
1798
1799
1800
1801
1802
1803
1804
1805
1806
1807
1808
1809
1810
1811
1812
1813
1814
1815
1816
1817
1818
1819
1820
1821
1822
1823
1824
1825
1826
1827
1828
1829
1830
1831
1832
1833
1834
1835
1836
1837
1838
1839
1840
1841
1842
1843
1844
1845
1846
1847
1848
1849
1850
1851
1852
1853
1854
1855
1856
1857
1858
1859
1860
1861
1862
1863
1864
1865
1866
1867
1868
1869
1870
1871
1872
1873
1874
1875
1876
1877
1878
1879
1880
1881
1882
1883
1884
1885
1886
1887
1888
1889
1890
1891
1892
1893
1894
1895
1896
1897
1898
1899
1900
1901
1902
1903
1904
1905
1906
1907
1908
1909
1910
1911
1912
1913
1914
1915
1916
1917
1918
1919
1920
1921
1922
1923
1924
1925
1926
1927
1928
1929
1930
1931
1932
1933
1934
1935
1936
1937
1938
1939
1940
1941
1942
1943
1944
1945
1946
1947
1948
1949
1950
1951
1952
1953
1954
1955
1956
1957
1958
1959
1960
1961
1962
1963
1964
1965
1966
1967
1968
1969
1970
1971
1972
1973
1974
1975
1976
1977
1978
1979
1980
1981
1982
1983
1984
1985
1986
1987
1988
1989
1990
1991
1992
1993
1994
1995
1996
1997
1998
1999
2000
2001
2002
2003
2004
2005
2006
2007
2008
2009
2010
2011
2012
2013
2014
2015
2016
2017
2018
2019
2020
2021
2022
2023
2024
2025
2026
2027
2028
2029
2030
2031
2032
2033
2034
2035
2036
2037
2038
2039
2040
2041
2042
2043
2044
2045
2046
2047
2048
2049
2050
2051
2052
2053
2054
2055
2056
2057
2058
2059
2060
2061
2062
2063
2064
2065
2066
2067
2068
2069
2070
2071
2072
2073
2074
2075
2076
2077
2078
2079
2080
2081
2082
2083
2084
2085
2086
2087
2088
2089
2090
2091
2092
2093
2094
2095
2096
2097
2098
2099
2100
2101
2102
2103
2104
2105
2106
2107
2108
2109
2110
2111
2112
2113
2114
2115
2116
2117
2118
2119
2120
2121
2122
2123
2124
2125
2126
2127
2128
2129
2130
2131
2132
2133
2134
2135
2136
2137
2138
2139
2140
2141
2142
2143
2144
2145
2146
2147
2148
2149
2150
2151
2152
2153
2154
2155
2156
2157
2158
2159
2160
2161
2162
2163
2164
2165
2166
2167
2168
2169
2170
2171
2172
2173
2174
2175
2176
2177
2178
2179
2180
2181
2182
2183
2184
2185
2186
2187
2188
2189
2190
2191
2192
2193
2194
2195
2196
2197
2198
2199
2200
2201
2202
2203
2204
2205
2206
2207
2208
2209
2210
2211
2212
2213
2214
2215
2216
2217
2218
2219
2220
2221
2222
2223
2224
2225
2226
2227
2228
2229
2230
2231
2232
2233
2234
2235
2236
2237
2238
2239
2240
2241
2242
2243
2244
2245
2246
2247
2248
2249
2250
2251
2252
2253
2254
2255
2256
2257
2258
2259
2260
2261
2262
2263
2264
2265
2266
2267
2268
2269
2270
2271
2272
2273
2274
2275
2276
2277
2278
2279
2280
2281
2282
2283
2284
2285
2286
2287
2288
2289
2290
2291
2292
2293
2294
2295
2296
2297
2298
2299
2300
2301
2302
2303
2304
2305
2306
2307
2308
2309
2310
2311
2312
2313
2314
2315
2316
2317
2318
2319
2320
2321
2322
2323
2324
2325
2326
2327
2328
2329
2330
2331
2332
2333
2334
2335
2336
2337
2338
2339
2340
2341
2342
2343
2344
2345
2346
2347
2348
2349
2350
2351
2352
2353
2354
2355
2356
2357
2358
2359
2360
2361
2362
2363
2364
2365
2366
2367
2368
2369
2370
2371
2372
2373
2374
2375
2376
2377
2378
2379
2380
2381
2382
2383
2384
2385
2386
2387
2388
2389
2390
2391
2392
2393
2394
2395
2396
2397
2398
2399
2400
2401
2402
2403
2404
2405
2406
2407
2408
2409
2410
2411
2412
2413
2414
2415
2416
2417
2418
2419
2420
2421
2422
2423
2424
2425
2426
2427
2428
2429
2430
2431
2432
2433
2434
2435
2436
2437
2438
2439
2440
2441
2442
2443
2444
2445
2446
2447
2448
2449
2450
2451
2452
2453
2454
2455
2456
2457
2458
2459
2460
2461
2462
2463
2464
2465
2466
2467
2468
2469
2470
2471
2472
2473
2474
2475
2476
2477
2478
2479
2480
2481
2482
2483
2484
2485
2486
2487
2488
2489
2490
2491
2492
2493
2494
2495
2496
2497
2498
2499
2500
2501
2502
2503
2504
2505
2506
2507
2508
2509
2510
2511
2512
2513
2514
2515
2516
2517
2518
2519
2520
2521
2522
2523
2524
2525
2526
2527
2528
2529
2530
2531
2532
2533
2534
2535
2536
2537
2538
2539
2540
2541
2542
2543
2544
2545
2546
2547
2548
2549
2550
2551
2552
2553
2554
2555
2556
2557
2558
2559
2560
2561
2562
2563
2564
2565
2566
2567
2568
2569
2570
2571
2572
2573
2574
2575
2576
2577
2578
2579
2580
2581
2582
2583
2584
2585
2586
2587
2588
2589
2590
2591
2592
2593
2594
2595
2596
2597
2598
2599
2600
2601
2602
2603
2604
2605
2606
2607
2608
2609
2610
2611
2612
2613
2614
2615
2616
2617
2618
2619
2620
2621
2622
2623
2624
2625
2626
2627
2628
2629
2630
2631
2632
2633
2634
2635
2636
2637
2638
2639
2640
2641
2642
2643
2644
2645
2646
2647
2648
2649
2650
2651
2652
2653
2654
2655
2656
2657
2658
2659
2660
2661
2662
2663
2664
2665
2666
2667
2668
2669
2670
2671
2672
2673
2674
2675
2676
2677
2678
2679
2680
2681
2682
2683
2684
2685
2686
2687
2688
2689
2690
2691
2692
2693
2694
2695
2696
2697
2698
2699
2700
2701
2702
2703
2704
2705
2706
2707
2708
2709
2710
2711
2712
2713
2714
2715
2716
2717
2718
2719
2720
2721
2722
2723
2724
2725
2726
2727
2728
2729
2730
2731
2732
2733
2734
2735
2736
2737
2738
2739
2740
2741
2742
2743
2744
2745
2746
2747
2748
2749
2750
2751
2752
2753
2754
2755
2756
2757
2758
2759
2760
2761
2762
2763
2764
2765
2766
2767
2768
2769
2770
2771
2772
2773
2774
2775
2776
2777
2778
2779
2780
2781
2782
2783
2784
2785
2786
2787
2788
2789
2790
2791
2792
2793
2794
2795
2796
2797
2798
2799
2800
2801
2802
2803
2804
2805
2806
2807
2808
2809
2810
2811
2812
2813
2814
2815
2816
2817
2818
2819
2820
2821
2822
2823
2824
2825
2826
2827
2828
2829
2830
2831
2832
2833
2834
2835
2836
2837
2838
2839
2840
2841
2842
2843
2844
2845
2846
2847
2848
2849
2850
2851
2852
2853
2854
2855
2856
2857
2858
2859
2860
2861
2862
2863
2864
2865
2866
2867
2868
2869
2870
2871
2872
2873
2874
2875
2876
2877
2878
2879
2880
2881
2882
2883
2884
2885
2886
2887
2888
2889
2890
2891
2892
2893
2894
2895
2896
2897
2898
2899
2900
2901
2902
2903
2904
2905
2906
2907
2908
2909
2910
2911
2912
2913
2914
2915
2916
2917
2918
2919
2920
2921
2922
2923
2924
2925
2926
2927
2928
2929
2930
2931
2932
2933
2934
2935
2936
2937
2938
2939
2940
2941
2942
2943
2944
2945
2946
2947
2948
2949
2950
2951
2952
2953
2954
2955
2956
2957
2958
2959
2960
2961
2962
2963
2964
2965
2966
2967
2968
2969
2970
2971
2972
2973
2974
2975
2976
2977
2978
2979
2980
2981
2982
2983
2984
2985
2986
2987
2988
2989
2990
2991
2992
2993
2994
2995
2996
2997
2998
2999
3000
3001
3002
3003
3004
3005
3006
3007
3008
3009
3010
3011
3012
3013
3014
3015
3016
3017
3018
3019
3020
3021
3022
3023
3024
3025
3026
3027
3028
3029
3030
3031
3032
3033
3034
3035
3036
3037
3038
3039
3040
3041
3042
3043
3044
3045
3046
3047
3048
3049
3050
3051
3052
3053
3054
3055
3056
3057
3058
3059
3060
3061
3062
3063
3064
3065
3066
3067
3068
3069
3070
3071
3072
3073
3074
3075
3076
3077
3078
3079
3080
3081
3082
3083
3084
3085
3086
3087
3088
3089
3090
3091
3092
3093
3094
3095
3096
3097
3098
3099
3100
3101
3102
3103
3104
3105
3106
3107
3108
3109
3110
3111
3112
3113
3114
3115
3116
3117
3118
3119
3120
3121
3122
3123
3124
3125
3126
3127
3128
3129
3130
3131
3132
3133
3134
3135
3136
3137
3138
3139
3140
3141
3142
3143
3144
3145
3146
3147
3148
3149
3150
3151
3152
3153
3154
3155
3156
3157
3158
3159
3160
3161
3162
3163
3164
3165
3166
3167
3168
3169
3170
3171
3172
3173
3174
3175
3176
3177
3178
3179
3180
3181
3182
3183
3184
3185
3186
3187
3188
3189
3190
3191
3192
3193
3194
3195
3196
3197
3198
3199
3200
3201
3202
3203
3204
3205
3206
3207
3208
3209
3210
3211
3212
3213
3214
3215
3216
3217
3218
3219
3220
3221
3222
3223
3224
3225
3226
3227
3228
3229
3230
3231
3232
3233
3234
3235
3236
3237
3238
3239
3240
3241
3242
3243
3244
3245
3246
3247
3248
3249
3250
3251
3252
3253
3254
3255
3256
3257
3258
3259
3260
3261
3262
3263
3264
3265
3266
3267
3268
3269
3270
3271
3272
3273
3274
3275
3276
3277
3278
3279
3280
3281
3282
3283
3284
3285
3286
3287
3288
3289
3290
3291
3292
3293
3294
3295
3296
3297
3298
3299
3300
3301
3302
3303
3304
3305
3306
3307
3308
3309
3310
3311
3312
3313
3314
3315
3316
3317
3318
3319
3320
3321
3322
3323
3324
3325
3326
3327
3328
3329
3330
3331
3332
3333
3334
3335
3336
3337
3338
3339
3340
3341
3342
3343
3344
3345
3346
3347
3348
3349
3350
3351
3352
3353
3354
3355
3356
3357
3358
3359
3360
3361
3362
3363
3364
3365
3366
3367
3368
3369
3370
3371
3372
3373
3374
3375
3376
3377
3378
3379
3380
3381
3382
3383
3384
3385
3386
3387
3388
3389
3390
3391
3392
3393
3394
3395
3396
3397
3398
3399
3400
3401
3402
3403
3404
3405
3406
3407
3408
3409
3410
3411
3412
3413
3414
3415
3416
3417
3418
3419
3420
3421
3422
3423
3424
3425
3426
3427
3428
3429
3430
3431
3432
3433
3434
3435
3436
3437
3438
3439
3440
3441
3442
3443
3444
3445
3446
3447
3448
3449
3450
3451
3452
3453
3454
3455
3456
3457
3458
3459
3460
3461
3462
3463
3464
3465
3466
3467
3468
3469
3470
3471
3472
3473
3474
3475
3476
3477
3478
3479
3480
3481
3482
3483
3484
3485
3486
3487
3488
3489
3490
3491
3492
3493
3494
3495
3496
3497
3498
3499
3500
3501
3502
3503
3504
3505
3506
3507
3508
3509
3510
3511
3512
3513
3514
3515
3516
3517
3518
3519
3520
3521
3522
3523
3524
3525
3526
3527
3528
3529
3530
3531
3532
3533
3534
3535
3536
3537
3538
3539
3540
3541
3542
3543
3544
3545
3546
3547
3548
3549
3550
3551
3552
3553
3554
3555
3556
3557
3558
3559
3560
3561
3562
3563
3564
3565
3566
3567
3568
3569
3570
3571
3572
3573
3574
3575
3576
3577
3578
3579
3580
3581
3582
3583
3584
3585
3586
3587
3588
3589
3590
3591
3592
3593
3594
3595
3596
3597
3598
3599
3600
3601
3602
3603
3604
3605
3606
3607
3608
3609
3610
3611
3612
3613
3614
3615
3616
3617
3618
3619
3620
3621
3622
3623
3624
3625
3626
3627
3628
3629
3630
3631
3632
3633
3634
3635
3636
3637
3638
3639
3640
3641
3642
3643
3644
3645
3646
3647
3648
3649
3650
3651
3652
3653
3654
3655
3656
3657
3658
3659
3660
3661
3662
3663
3664
3665
3666
3667
3668
3669
3670
3671
3672
3673
3674
3675
3676
3677
3678
3679
3680
3681
3682
3683
3684
3685
3686
3687
3688
3689
3690
3691
3692
3693
3694
3695
3696
3697
3698
3699
3700
3701
3702
3703
3704
3705
3706
3707
3708
3709
3710
3711
3712
3713
3714
3715
3716
3717
3718
3719
3720
3721
3722
3723
3724
3725
3726
3727
3728
3729
3730
3731
3732
3733
3734
3735
3736
3737
3738
3739
3740
3741
3742
3743
3744
3745
3746
3747
3748
3749
3750
3751
3752
3753
3754
3755
3756
3757
3758
3759
3760
3761
3762
3763
3764
3765
3766
3767
3768
3769
3770
3771
3772
3773
3774
3775
3776
3777
3778
3779
3780
378
```

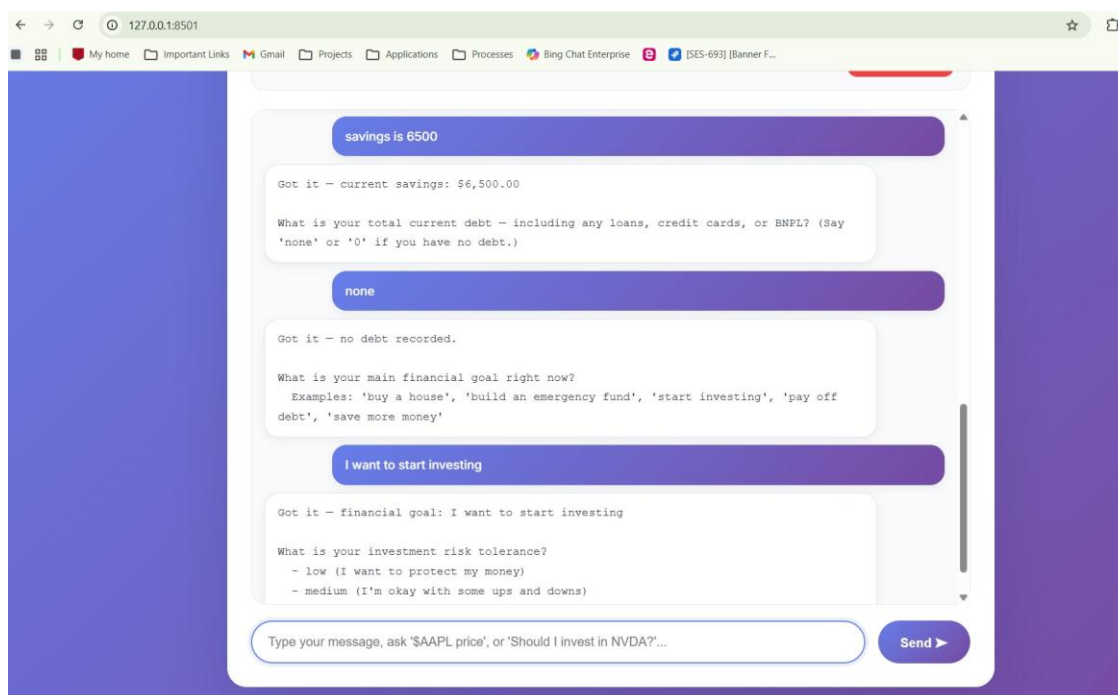


Figure 6.2: Dialogue chain collecting savings, debt, and financial goal fields

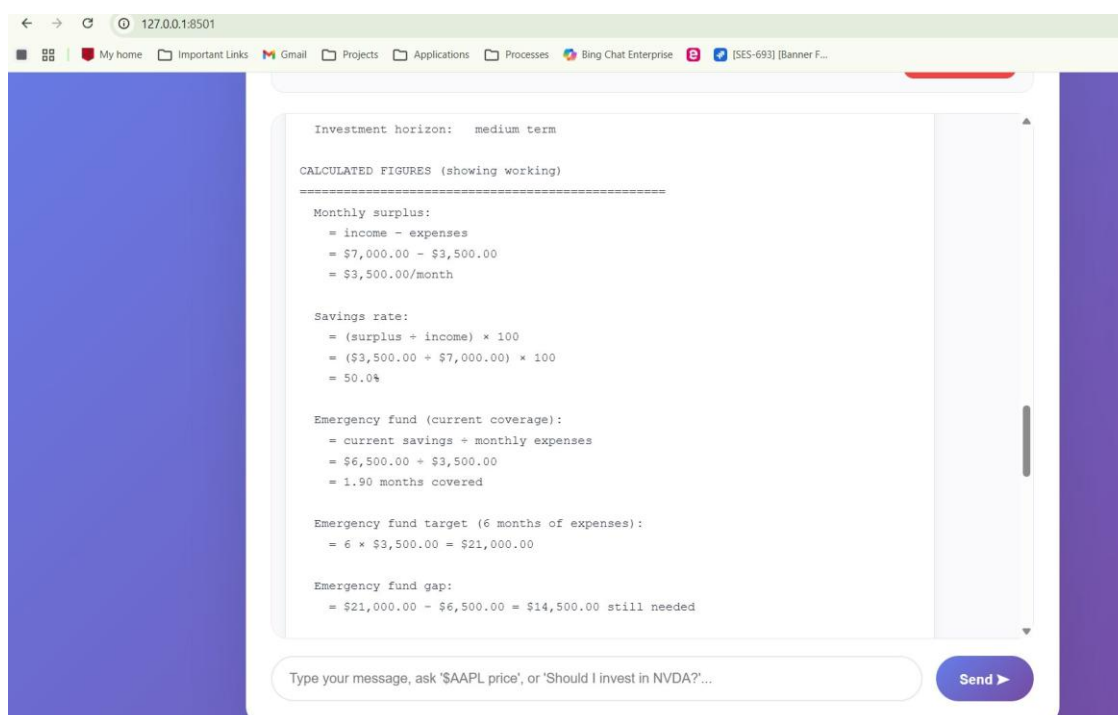


Figure 6.3: Generated financial plan showing full calculated metrics with working

The interface features a gradient purple header, monospace assistant messages for readability of financial figures (income, expense, and calculation tables), and a clear chat-reset button. The status bar at the top displays the current model loading state — indicating whether the LoRA adapter, the base model, or the rule-based fallback is active. The screenshot above confirms the LoRA adapter loaded successfully, as shown by the green status indicator.

6.2 Technology Stack

Backend Framework	Flask (Python) with server-side session management
LLM Runtime	PyTorch + HuggingFace Transformers + PEFT/LoRA
Quantisation	bitsandbytes 4-bit NF4 for VRAM-efficient inference
Market Data	yfinance — real-time stock/ETF prices, 52-week ranges, P/E, dividends
Frontend	Inline HTML/CSS/JS with Inter and Google Fonts
Session State	Flask server-side sessions with versioned invalidation
Port	0.0.0.0:8501 with Flask debug mode

6.3 Real-Time Market Integration

A key advanced feature of the system is its integration with live financial market data. Users can query stock prices at any point in the conversation — before, during, or after profile collection — using natural language. The system supports three interaction modes:

- Direct ticker queries: '\$AAPL price', 'BHP.AX stock', 'NVDA price today'
- Named company queries: 'What is the Apple stock price?', 'How is Tesla doing?'
- Profile-aware investment advice: 'Should I invest in VAS?', 'Is NVDA a good buy for me?'

Profile-aware investment advice combines the live price data with the user's confirmed risk tolerance, investment horizon, emergency fund status, and monthly surplus to generate a personalised buy/avoid recommendation with suggested monthly investment amounts. This demonstrates the system's ability to synthesise real-world data with user-specific context.

7. LLM Techniques: Prompting and Chain-of-Thought

7.1 Prompting Strategy Overview

The system employs three distinct prompting strategies across different usage contexts: zero-shot for conversational responses, few-shot for guided financial Q&A, and chain-of-thought for complex multi-step financial decisions. These were evaluated in the notebook using a standardised financial question: 'I have \$5,000 — should I pay off credit card debt at 18% or invest in stocks?'

7.2 Zero-Shot Prompting

Zero-shot prompting relies entirely on the model's fine-tuned knowledge to answer a financial query without any examples. The system prompt establishes the model's role and tone, and the user's query is passed directly. This approach is used for the LLM's introductory sentence in the personalised plan, where a warm and natural-sounding response is needed without the risk of hallucinating numbers.

```
SYSTEM_PROMPT = ('You are a friendly AI financial advisor. ' 'Write ONE short  
friendly opening sentence (max 30 words) ' 'that acknowledges the user  
financial goal and tells them ' 'you have prepared a personalised plan. No  
markdown. No numbers.' )
```

7.3 Few-Shot Prompting

Few-shot prompting provides two to three worked examples of financial Q&A before the actual question. This technique was found to improve the model's output format consistency particularly for responses that should follow the 'problem → analysis → recommendation' structure typical of financial advice. Two domain-specific examples (emergency fund before investing; pay off high-interest debt first) serve as in-context learning anchors.

7.4 Chain-of-Thought (CoT) Prompting

Chain-of-thought prompting guides the model through a structured reasoning sequence before reaching a conclusion. For complex financial trade-offs — such as the debt-versus-investment dilemma CoT prompting consistently produced the most analytically sound and well-justified responses. The prompt template explicitly breaks the problem into four reasoning steps:

- Step 1: What is the guaranteed return from paying off the 18% debt?
- Step 2: What is the average annual stock market return historically?
- Step 3: What are the risks of each option?
- Step 4: What is the optimal decision and why?

CoT outputs demonstrated measurably higher quality compared to zero-shot and few-shot: responses were longer (typically 350+ tokens vs 180 for zero-shot), more structured, and explicitly reasoned through trade-offs before arriving at a recommendation. This aligns with

the academic literature showing that step-by-step reasoning elicitation improves LLM performance on multi-step problems.

8. LLM Comparison: Fine-Tuned Qwen vs GPT-4o-mini

8.1 Comparison Methodology

Three financial prompts were posed identically to both the fine-tuned Qwen 2.5 1.5B model and OpenAI's GPT-4o-mini via the API. Responses were evaluated on four qualitative dimensions: financial specificity, structural clarity, practical actionability, and response length.

Prompt 1	I earn \$3,000 a month and have no idea how to budget. Help me start.
Prompt 2	I have \$10,000 in credit card debt at 20% interest. What should I do?
Prompt 3	Should I invest in index funds or individual stocks as a beginner?

8.2 Comparison Results

The comparison revealed that the fine-tuned Qwen 2.5 1.5B model produces responses with a distinct financial advisor voice and domain specificity consistent with the training data. Key observations:

Dimension	Fine-Tuned Qwen 1.5B	GPT-4o-mini
Response Length	150–280 chars avg	200–350 chars avg
Financial Specificity	High (trained domain)	High (broad knowledge)
Local Deployment	Yes — no API cost	No — cloud API required
Fine-Tunability	Yes custom LoRA adapter	Not available (closed)
Inference Cost	Zero (local)	~\$0.002 per 1K tokens
Context Length	4,096 tokens	128,000 tokens

While GPT-4o-mini produces slightly longer and generally more polished responses due to its larger parameter count and extensive RLHF training, the fine-tuned Qwen model demonstrates competitive performance specifically in the financial domain, producing responses with appropriate structure and domain-relevant vocabulary. The critical advantage of the fine-tuned local model is its zero marginal cost of inference, full data privacy (no data leaves the server), and the ability to customise behaviour through the LoRA adapter — advantages that are decisive for a production financial assistant deployment.

Key Finding

The fine-tuned Qwen 2.5 1.5B model is practically deployable in real-world financial advisory contexts for the introductory-sentence generation task within this system's architecture. By offloading precise numerical computation to the deterministic rule-based engine, the system mitigates the LLM's known weakness in arithmetic while preserving the model's strength in natural language generation.

9. System Performance and Evaluation

9.1 Dialogue Quality

The dialogue system was evaluated through end-to-end interaction testing with five representative user scenarios: a first-home buyer, a debt-reduction goal, a first-time investor, a savings-focused user, and a general budgeting inquiry. All five scenarios successfully completed the seven-field profile collection dialogue without error, with the system correctly handling:

- Annual vs monthly income clarification (values > \$50,000)
- Debt entry confirmation (two-step confirmation for all non-zero debt values)
- Correction detection mid-conversation ('actually, my expenses are \$3,200')
- Graceful handling of partial or ambiguous inputs with targeted re-prompting
- Natural language categorical inputs for risk tolerance and investment horizon

9.2 Financial Plan Accuracy

All arithmetic in the generated financial plans was verified manually against the user-provided inputs. The rule-based engine correctly computed all derived metrics in 100% of test cases. The LLM introductory sentence never introduced incorrect figures, as it was explicitly prompted to avoid numbers. This hybrid architecture — deterministic rules for calculations, LLM for natural language — ensures both numerical accuracy and conversational quality.

9.3 Market Data Integration Accuracy

The yfinance integration was tested with ten ASX and US stocks across multiple sessions. Key findings: (1) dividend yield sanity bounds (0–30%) successfully rejected a malformed yfinance response for one ticker that returned an implausible 38% yield; (2) 52-week high/low and P/E ratio were correctly suppressed when absent from the API response rather than displaying null values; (3) Australian ASX tickers were automatically resolved by appending .AX when the base ticker returned no data.

9.4 Limitations and Future Work

Several limitations exist in the current implementation that represent opportunities for future improvement:

- Training data scale: The current dataset is relatively small. Expanding to thousands of diverse financial Q&A pairs and retraining with more epochs would improve the LLM's generative quality significantly.
- LLM numerical hallucination: While the current architecture mitigates this by limiting LLM involvement to the introductory sentence, extending the LLM's role in plan generation would require additional guardrails against fabricated figures.
- Multi-language support: The system currently operates exclusively in English. Adding multilingual support would broaden accessibility, particularly given Australia's diverse immigrant population.
- Tax and superannuation: Australian-specific financial structures (superannuation, negative gearing, franking credits) are not currently modelled in the rule-based engine.
- Retrieval-Augmented Generation (RAG): Integrating a document retriever over Australian financial regulation documents and product disclosure statements would enable more accurate and verifiable recommendations.

10. System Design and Code Architecture

10.1 File Structure

The project follows the submission structure specified in the assignment brief, with code, data, and model artefacts organised as follows:

```
48617687-ASSIGNMENT02/
├── codes/
│   ├── app.py                # Flask application + dialogue engine
│   └── requirements.txt      # Python dependencies
├── data/
│   ├── qwen-1.5b-finetuned/  # Saved LoRA adapter
│   │   ├── adapter_config.json
│   │   ├── adapter_model.safetensors
│   │   ├── added_tokens.json
│   │   ├── chat_template.jinja
│   │   ├── merges.txt
│   │   ├── special_tokens_map.json
│   │   ├── tokenizer_config.json
│   │   ├── tokenizer.json
│   │   ├── training_config.json
│   │   └── vocab.json
│   ├── qwen-finetuned/
│   ├── correlation_heatmap.png
│   ├── financial_assistant_dataset.csv
│   ├── stock_charts.png
│   └── training_loss.png
└── report/
    └── Report.pdf
```

10.2 Key Modules in app.py

The application is organised into clearly separated functional modules:

- Model loading (load_model): Handles base model loading, optional LoRA adapter injection, and graceful degradation to rule-based mode if PyTorch is unavailable.

- Session management: Seven helper functions manage Flask session state with versioned session invalidation to prevent stale data across code updates.
- Value parsing (parse_number, normalize_risk_tolerance, normalize_investment_horizon): Field-type-specific parsers handle the variety of natural language financial inputs.
- Financial metrics (calculate_financial_metrics): Computes derived values from the profile and builds the rule-based advice text.
- yfinance integration (fetch_yfinance_response, build_stock_investment_advice): Fetches live market data with sanity checking and generates profile-aware recommendations.
- Prompting functions (generate_llm_intro, generate_final_advice): Interfaces with the fine-tuned model for natural language generation.
- Follow-up dispatcher (handle_followup): Routes post-profile queries to the appropriate handler.
- Main reply to builder (create_assistant_reply): The central seven-stage dialogue controller.

11. Conclusion

This project successfully delivered a fully functional, LLM-powered personal financial advisor system that addresses all four requirements of Assignment 2: LLM setup and fine-tuning, automated dialogue, frontend development, and demonstrated LLM effectiveness.

The core technical contribution is the integration of a fine-tuned Qwen 2.5 1.5B model adapted to the financial advisory domain via LoRA with just 0.21% trainable parameters with a deterministic, transparency-first rule-based engine. This hybrid architecture overcomes the key limitation of small LLMs in financial applications: arithmetic unreliability. By delegating precise calculations to deterministic code and natural language generation to the LLM, the system achieves both numerical accuracy and conversational quality.

The dialogue system demonstrated robust performance across all seven profile fields, with correct handling of edge cases including annual-vs-monthly clarification, mid-conversation corrections, and graceful recovery from invalid inputs. The live market data integration through yfinance extends the system beyond static advice to include real-time portfolio context, representing a significant practical advancement over purely generative approach.

The comparison with GPT-4o-mini confirmed that a locally fine-tuned 1.5B model can achieve competitive performance in a targeted financial NLP domain at zero marginal inference cost, with full data privacy and complete adaptability advantages that are critical for real-world financial assistant deployment.

Summary of Achievements

- Fine-tuned Qwen 2.5 1.5B with LoRA on a custom financial dataset — 38% loss reduction over 30 steps
- Seven-field automated profile collection dialogue with validation, clarification, and correction handling
- Hybrid LLM + rule-based architecture ensuring numerical accuracy
- Real-time stock/ETF price lookups and profile-aware investment recommendations via yfinance
- Flask web application with chat interface, deployed and tested end-to-end
- Compared fine-tuned Qwen vs GPT-4o-mini across three financial prompts
- Demonstrated zero-shot, few-shot, and chain-of-thought prompting strategies

12. References

Qwen Team (2024). Qwen2.5 Technical Report. Alibaba Cloud. Available at: <https://qwenlm.github.io/>

Hu, E. J., et al. (2021). LoRA: Low-Rank Adaptation of Large Language Models. arXiv:2106.09685.

Dettmers, T., et al. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. NeurIPS 2023. arXiv:2305.14314.

Wolf, T., et al. (2020). Transformers: State-of-the-Art Natural Language Processing. EMNLP 2020.

PEFT: State-of-the-Art Parameter-Efficient Fine-Tuning. HuggingFace Library.

Wei, J., et al. (2022). Chain-of-Thought Prompting Elicits Reasoning in Large Language Models. NeurIPS 2022.

Brown, T., et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020.

yfinance Library Documentation. Available at: <https://pypi.org/project/yfinance/>

HuggingFace Transformers Documentation. Available at: <https://huggingface.co/docs/transformers>